# Frontend Developer Intern

Assignment 1 — Frontend Fundamentals Assessment

| | |
|---|---|
| **Submitted by** | Sanchit Upneja |
| **Email** | upnejasanchitq@gmail.com |
| **Date** | April 03, 2025 |
| **Assignment** | Frontend Fundamentals — Theoretical Assessment |

I've been learning frontend development for about 8 months, primarily building projects with React and vanilla JS. I've worked on a weather tracker app, a smart event dashboard, and a small tic tac toe game. This document contains my answers to the assignment questions — I've tried to explain things in my own words and include code snippets where they help clarify my thinking.

## Section 1 — HTML5 Semantics

**Q1.**
**What are semantic HTML elements and why are they important?**

Semantic elements are HTML tags that tell you *what the content is*, not just how it looks. Tags like `<header>`, `<nav>`, `<main>`, `<article>`, `<section>`, `<aside>`, and `<footer>` all carry meaning. Compare that to a plain `<div>` which doesn't say anything about what's inside it.

There are three main reasons I think they matter:

- **Accessibility —** Screen readers use these landmarks to help visually impaired users jump between sections of a page. Without them, a screen reader user has to listen through everything linearly.
- **SEO —** Search engines give more weight to content inside semantic tags like `<article>` and `<h1>`. A page full of generic divs is harder for crawlers to parse.
- **Readability —** When I come back to code I wrote a month ago, semantic HTML tells the story of the page structure at a glance. I don't have to trace through class names to understand the layout.

```
<!-- What I try to avoid -->

<div class="top-bar">

<div class="nav-links"> ... </div>

</div>

<div class="page-body">

<div class="blog-post"> ... </div>

</div>


<!-- What I aim for instead -->

<header>

<nav aria-label="Main navigation"> ... </nav>

</header>

<main>

<article>

<h1>Post title</h1>
```

```
<p>Content here...</p>

</article>

</main>

<footer> ... </footer>
```

**Q2.**
**Explain the difference between <section>, <article>, , and .**

I find the easiest way to remember this is to ask: *"Does this make sense on its own?"*

- **<article>** — self-contained content that you could lift out and publish elsewhere. A blog post, a news story, a product card. If you can RSS-feed it, it's probably an article.
- **<section>** — a thematic group within a page. It should always have a heading. I use it to divide a long page into named parts (e.g. "Features", "Pricing", "FAQ").
- ** ** — no semantic meaning at all. I only reach for it when I genuinely need a wrapper for CSS/JS and no semantic element fits. It's my last resort, not my first.
- **** — same idea as div but inline. Useful for wrapping a word or phrase to apply a style or hook in JavaScript without breaking the text flow.

**Q3.**
**What are the key differences between HTML4 and HTML5? Give at least 5.**

The ones I find most relevant in day-to-day work:

- **Simpler doctype:** HTML5 uses just `<!DOCTYPE html>`. The old HTML4 DOCTYPE was a long, cryptic SGML string nobody memorised.
- **Native video & audio:** Before HTML5 you needed Flash for media. Now `<video>` and `<audio>` are built into the browser.
- **Semantic elements:** header, footer, nav, article, section, aside — none of these existed in HTML4.
- **Better form inputs:** type="email", type="date", type="range" with built-in validation. I no longer need a date-picker library for simple cases.
- **localStorage / sessionStorage:** Client-side storage without abusing cookies.
- **canvas element:** Allows drawing graphics directly via JavaScript — used for charts, games, image manipulation.
- **Deprecated presentational tags removed:** <font>, , <strike> — HTML5 pushes all presentation to CSS where it belongs.

---

## Section 2 — CSS3 Layouts & Responsiveness

**Q4.**
**Compare Flexbox and Grid — when do you use each?**

Honestly this clicked for me when I started thinking of it this way: **Flexbox is for one direction at a time, Grid is for two directions at once.**

I use **Flexbox** when I'm aligning things in a row or a column — navbars, button groups, centering a card in the middle of the screen, laying out form fields. It's great when the content itself should drive how much space things take up.

I use **Grid** for the overall page layout — where I need rows AND columns at the same time. A sidebar + main content + header + footer layout is the classic case. I also use it for image galleries where I want a strict column count.

```
/* Flexbox — great for a navbar */

.navbar {

display: flex;

align-items: center;

justify-content: space-between;

gap: 1rem;

}


/* Grid — page shell layout */

.layout {

display: grid;

grid-template-columns: 250px 1fr;

grid-template-rows: 60px 1fr 50px;

grid-template-areas:

"header header"

"sidebar main"

"footer footer";

min-height: 100vh;

}
```

```
.header { grid-area: header; }

.sidebar { grid-area: sidebar; }

.main { grid-area: main; }
```

**Q5.**

### Explain the CSS Box Model. How does box-sizing affect it?

Every HTML element is basically a box with four layers. From inside out: **content** → **padding** → **border** → **margin**.

The part that tripped me up early on was how **box-sizing** changes what width/height actually mean:

- **box-sizing: content-box** (the default) — width only measures the content area. Padding and border get added on top. So a 200px wide element with 20px padding ends up being 240px wide. Super confusing.
- **box-sizing: border-box** — width includes padding and border. A 200px element stays 200px. This is what I always want, so I apply it globally:

```
*, *::before, *::after {

box-sizing: border-box; /* fixes the box model for the entire project */

}
```

Almost every CSS reset and framework (Tailwind, Bootstrap) does this. Once I understood why, I added it to every project from day one.

**Q6.**

### What are CSS custom properties (variables) and how do they help with theming?

CSS variables (officially called custom properties) let you store values in one place and reuse them everywhere. You declare them on `:root` and use them with `var()`:

```
:root {

--color-primary: #2563eb;

--color-bg: #ffffff;

--color-text: #1e293b;

--radius: 8px;

--spacing-md: 1rem;

}
```

```
.button {
background: var(--color-primary);
border-radius: var(--radius);
padding: var(--spacing-md);
}
```

The real power for theming is that you can override the whole set in one block. For dark mode I just do:

```
@media (prefers-color-scheme: dark) {
:root {
--color-bg: #0f172a;
--color-text: #f1f5f9;
--color-primary: #60a5fa;
}
}
/* Every component that uses these variables automatically updates — no extra code */
```

I used this approach on my portfolio site and switching themes was literally a one-liner. Before I knew about CSS variables I was doing find-replace on hex codes, which was painful.

**Q7.**

**How does responsive design work? Explain media queries and mobile-first approach.**

Responsive design means the layout adapts to fit different screen sizes instead of having separate mobile and desktop versions.

**Media queries** apply styles conditionally. The most common usage is based on viewport width:

```
/* Mobile-first: base styles target small screens */
.card {
padding: 12px;
font-size: 14px;
```

```
}

/* Tablet: enhance at 640px and up */
@media (min-width: 640px) {
.card { padding: 20px; font-size: 16px; }
}

/* Desktop: enhance further at 1024px and up */
@media (min-width: 1024px) {
.card { padding: 28px; font-size: 18px; max-width: 480px; }
}
```

The **mobile-first** approach means writing the base styles for the smallest screen and then adding overrides for larger screens using `min-width`. The alternative (desktop-first with `max-width`) technically works but mobile devices end up downloading desktop CSS and overriding it, which wastes bandwidth. Mobile-first is more performant and honestly forces you to think about what actually matters on a small screen first.

---

## Section 3 — JavaScript (ES6+)

**Q8.**
### Explain var, let, and const — differences in scope and hoisting.

This is one of those things that confused me for a while, so let me try to explain it clearly.

**var** is the old way. It has two big problems:

- It's function-scoped, not block-scoped — so a var declared inside an if block leaks out of it
- It gets hoisted and initialised as undefined, which means you can accidentally use it before it's declared without getting an error — just a silent bug

**let** and **const** are both block-scoped. They are hoisted but NOT initialised — accessing them before their declaration throws a ReferenceError (called the Temporal Dead Zone). This is actually better because it makes the bug obvious.

```
// var — leaks out of block
if (true) { var x = 10; }
```

```
console.log(x); // 10 — this works, which is surprising


// let — stays in block

if (true) { let y = 10; }

console.log(y); // ReferenceError — expected behaviour


// const — can't be reassigned (but object properties CAN change)

const user = { name: 'Rahul' };

user.name = 'Raj'; // fine — we're mutating, not reassigning

user = {}; // TypeError — can't reassign the binding
```

My rule: **use const by default**. Switch to let only when you know the value will be reassigned. Never use var in new code.

**Q9.**
### What is the difference between == and ===? Why prefer ===?

`==` does **type coercion** before comparing — it tries to convert both values to the same type, which leads to some really unintuitive results. `===` (strict equality) checks value AND type, no conversion.

```
// == surprises that have bitten me before:

0 == false // true — false coerces to 0

'' == false // true — empty string coerces to 0

null == undefined // true — special case in the spec

1 == '1' // true — string coerces to number


// === is predictable:

0 === false // false — different types

1 === '1' // false — number vs string

null === undefined // false — different types
```

I always use `===`. The only time I'd consciously use `==` is checking for null/undefined simultaneously (`value == null`), and even then I prefer explicit checks. ESLint's eqeqeq rule enforces strict equality

across a codebase automatically.

**Q10.**

### Explain Promises, async/await, and the event loop.

**Event loop (simplified):**

JavaScript is single-threaded. There's a call stack where code runs. When an async operation (like a network request) finishes, its callback gets queued. The event loop keeps checking: 'is the stack empty? If yes, pull the next item from the queue.' Microtasks (Promises) have a separate queue that gets drained completely before the next macrotask (setTimeout etc.) runs.

**Promises:**

A Promise wraps an async operation and represents a value that isn't available yet. It's in one of three states: pending, fulfilled, or rejected. You handle the result with `.then()` and errors with `.catch()`.

**async/await:**

This is syntactic sugar over Promises that makes async code read almost like synchronous code. An async function always returns a Promise. `await` pauses execution inside the function until the Promise settles — without blocking the main thread.

```
// Callback hell (old way — hard to read)

fetchUser(id, function(user) {

fetchPosts(user.id, function(posts) {

render(posts);

});

});


// async/await (modern — much cleaner)

async function loadUserPosts(id) {

try {

const user = await fetchUser(id);

const posts = await fetchPosts(user.id);

render(posts);

} catch (err) {

showError(err.message);
```

```
}

}


// Running two independent requests in parallel (don't await one by one here)

const [user, config] = await Promise.all([fetchUser(id), fetchConfig()]);
```

**Q11.**

### What are closures? Give a practical example.

A closure happens when a function 'remembers' the variables from its outer scope even after that outer function has finished running. The inner function closes over those variables.

Here's a simple example I actually used when building a counter component:

```
function makeCounter(initialValue = 0) {

let count = initialValue; // this variable lives on even after makeCounter returns


return {

increment() { return ++count; },

decrement() { return --count; },

reset() { count = initialValue; },

value() { return count; },

};

}


const counter = makeCounter(5);

counter.increment(); // 6

counter.increment(); // 7

counter.decrement(); // 6

// 'count' is private — there's no way to access it directly from outside
```

Closures are also how React's useState works under the hood — the setter function closes over the current state value. And they're used in event listeners that need to remember some data when the event fires.

**Q12.**

### Arrow functions vs regular functions — key differences.

The biggest difference that actually matters in practice is **how they handle 'this'**.

- **Regular functions** get their own 'this' — it depends on HOW the function is called. In a class method called via setInterval, 'this' becomes undefined or the global object.
- **Arrow functions** don't have their own 'this' — they inherit it from wherever they were written (the lexical scope). This makes them predictable inside class methods and callbacks.

```
class Timer {

constructor() {

this.count = 0;

}


// Regular function — 'this' gets re-bound by setInterval, breaks

startBad() {

setInterval(function() {

this.count++; // 'this' is undefined here!

}, 1000);

}


// Arrow function — 'this' refers to the Timer instance, works correctly

start() {

setInterval(() => {

this.count++; // 'this' is the Timer instance

}, 1000);

}
```

```
}
```

Other differences worth knowing:

- Arrow functions can't be used as constructors (can't call with new)
- Arrow functions have no 'arguments' object — use rest params (...args) instead
- Short syntax allows implicit return for single expressions: `const double = n => n * 2`

---

## Section 4 — React.js Fundamentals

**Q13.**

### Explain the Virtual DOM and how React uses it.

The Virtual DOM is a lightweight JavaScript copy of the real DOM kept in memory. When something changes in a React component, React doesn't immediately touch the real DOM (which is slow). Instead:

- It builds a new Virtual DOM tree for the changed component
- It compares (diffs) the new tree against the previous one to find exactly what changed
- It batches those minimal changes and applies them to the real DOM in one go

This matters because direct DOM operations are expensive — they can trigger reflows and repaints. By batching minimal diffs, React keeps UIs fast even with lots of state changes. React 18 goes further with concurrent rendering, which can interrupt and prioritise render work so the UI never freezes.

**Q14.**

### What are React hooks? Explain useState, useEffect, useCallback, and useMemo.

Hooks let function components do things that only class components could do before — like manage state and run side effects.

```
import { useState, useEffect, useCallback, useMemo } from 'react';


function SearchablePosts({ categoryId }) {

// useState — stores local component state

// Re-renders the component whenever value changes

const [posts, setPosts] = useState([]);

const [search, setSearch] = useState('');

const [loading, setLoading] = useState(true);


// useEffect — runs side effects after render

// Empty array [] = run once on mount

// [categoryId] = re-run whenever categoryId changes

useEffect(() => {

setLoading(true);

fetchPosts(categoryId)

.then(data => setPosts(data))
```

```
.finally(() => setLoading(false));

}, [categoryId]);


// useMemo — cache the result of an expensive calculation

// Recalculates only when posts or search changes

const filteredPosts = useMemo(

() => posts.filter(p =>

p.title.toLowerCase().includes(search.toLowerCase())

),

[posts, search]

);


// useCallback — returns a memoized function

// Useful when passing callbacks to child components to avoid re-renders

const handleLike = useCallback((postId) => {

likePost(postId);

}, []); // function never changes


if (loading) return <p>Loading...</p>;

return ( /* JSX here */ );

}
```

**Q15.**

### What is the React component lifecycle? How do hooks map to it?

In class components, lifecycle methods are explicit. With hooks, the same phases are handled through `useEffect`:

```
// Mount — run once after component first appears
```

```
useEffect(() => {

fetchData();

}, []); // empty array = no deps = only on mount


// Update — run when specific values change

useEffect(() => {

updateChart(data);

}, [data]); // runs on mount + whenever 'data' changes


// Unmount — cleanup function runs before component is removed

useEffect(() => {

const subscription = subscribe(channel);

return () => subscription.unsubscribe(); // cleanup

}, [channel]);
```

The key insight: a single useEffect can handle mount, update, AND unmount. The return function is always the cleanup. This felt weird to me at first coming from class components, but it actually makes sense — effects and their cleanup are colocated.

**Q16.**

### What is prop drilling and how does Context API solve it?

**Prop drilling** is when you pass a prop through multiple layers of components just to get it to a deeply nested child — even though the middle components don't use it at all. It becomes a real problem when you need to change the prop because you have to touch every component in the chain.

```
// Without Context — prop drilling through App > Layout > Sidebar > UserMenu

function App() {

const [user, setUser] = useState(null);

return <Layout user={user} />; // Layout doesn't need user

}
```

```
function Layout({ user }) {
return <Sidebar user={user} />; // Sidebar doesn't need it either
}
function Sidebar({ user }) {
return <UserMenu user={user} />; // FINALLY used here
}

// With Context — skip the middle components entirely
const UserContext = React.createContext(null);

function App() {
const [user] = useState(null);
return (
<UserContext.Provider value={user}>
<Layout /> {/* doesn't receive user prop */}
</UserContext.Provider>
);
}

function UserMenu() {
const user = useContext(UserContext); // grabs it directly
return <div>Hello, {user?.name}</div>;
}
```

Context is great for things like auth state, themes, or language settings that are needed across many components. For heavier state management with more complex interactions, I'd reach for something like Zustand — it's simpler than Redux but more scalable than raw Context.

## Section 5 — State Management

**Q17.**
### Compare Redux, Zustand, and React Query.

I think about state management in two separate categories: **client state** (UI data that lives in memory) and **server state** (data fetched from an API that needs caching and syncing). Different tools are built for each.

**Redux Toolkit — client state, large scale:**

Best for complex applications where multiple parts of the app need to share and react to the same state. The strict unidirectional data flow makes bugs easier to trace. Redux DevTools for time-travel debugging is genuinely useful on bigger teams. The downside is it's verbose — even with Redux Toolkit simplifying the old boilerplate.

**Zustand — client state, simpler:**

This is what I'd pick for most projects. It's minimal — you define a store in about 10 lines, no reducers or actions required. Components subscribe to only the slices they care about, so re-renders are efficient by default.

**React Query (TanStack Query) — server state:**

A completely different kind of tool. It handles fetching, caching, background refetching, and stale data for API calls. Things that used to require a lot of manual useEffect and state management just work. I'd pair React Query with Zustand — React Query for server data, Zustand for UI state.

**Q18.**
### Explain the difference between local, global, server, and URL state.

- **Local state:** Only one component cares about it. A modal being open or closed, an input field's current value. useState is perfect here. No need to go global.
- **Global (client) state:** Multiple unrelated components need the same data. The logged-in user, the current theme, a shopping cart. This is what Context/Zustand/Redux is for.
- **Server state:** Data that lives on a backend. It needs to be fetched, kept fresh, and handled for loading/error states. React Query or SWR is built exactly for this.
- **URL state:** State encoded in the URL — search query, current page, filters. It survives page refresh and can be shared via link. Managed via the router (useSearchParams in React Router). I find this underused — a lot of things that go into useState should actually be in the URL.

---

## Section 6 — Performance Optimization

**Q19.**
### Explain code splitting, lazy loading, and their impact on Core Web Vitals.

**Code splitting** breaks the JavaScript bundle into smaller chunks. Instead of loading all the JS for every page upfront, you only load what's needed for the current page. **Lazy loading** defers loading of non-critical resources until they're actually needed (like when the user scrolls to them).

```
import { lazy, Suspense } from 'react';


// Each page becomes a separate bundle — loaded only when navigated to

const Dashboard = lazy(() => import('./pages/Dashboard'));

const Settings = lazy(() => import('./pages/Settings'));


function App() {

return (

<Suspense fallback={<div>Loading...</div>}>

<Routes>

<Route path="/dashboard" element={<Dashboard />} />

<Route path="/settings" element={<Settings />} />

</Routes>

</Suspense>

);

}


// Images below the fold — browser loads them only when near viewport

<img src="photo.jpg" loading="lazy" width="800" height="600" alt="..." />
```

**Impact on Core Web Vitals:**

- **LCP (Largest Contentful Paint)** improves because the initial JS bundle is smaller, so the page renders faster
- **INP (Interaction to Next Paint)** improves because less JS parsing on load means the main thread is free sooner
- **CLS (Cumulative Layout Shift)** — lazy-loaded images can actually hurt CLS if you don't set explicit width/height. Always set dimensions on images.

**Q20.**

### What is memoization in React? When is it appropriate?

Memoization caches the result of a computation so it doesn't re-run when inputs haven't changed.

```
// React.memo — skip re-rendering a component if its props haven't changed

const UserCard = React.memo(function UserCard({ name, avatar }) {

return <div><img src={avatar} />{name}</div>;

});


// useMemo — cache an expensive computed value

const expensiveList = useMemo(

() => rawData.filter(item => item.active).sort((a,b) => b.score - a.score),

[rawData] // only recalculate when rawData changes

);


// useCallback — stable function reference so child components don't re-render needlessly

const handleSubmit = useCallback((formData) => {

submitForm(formData);

}, []);
```

The honest caveat: **don't over-memoize.** Every useMemo and useCallback adds overhead — React still has to check if the dependencies changed. I only add memoization when the React DevTools Profiler shows actual unnecessary renders causing visible performance issues. Premature optimization here can make code harder to read for no real benefit.

**Q21.**

### List 5 frontend performance best practices.

- **Use next-gen image formats (WebP/AVIF)** and always set explicit width and height to prevent layout shift. Use responsive srcset for different screen sizes.
- **Defer non-critical JavaScript** with the defer or async attribute. Scripts block HTML parsing by default — defer moves execution to after parsing is done.
- **Implement proper caching** with content-hashed filenames for static assets and appropriate Cache-Control headers. Static files should be cached for a long time since the hash changes with content.
- **Reduce bundle size** — use tree shaking, audit dependencies, and avoid importing entire libraries when you only need one function (e.g., import debounce from 'lodash/debounce' not the whole lodash).

- **Virtualise long lists** — rendering a 10,000-row table destroys performance. Libraries like @tanstack/virtual render only the visible rows.
- **Debounce event handlers** — search inputs, resize handlers, and scroll events can fire hundreds of times per second. Debounce/throttle to only run the expensive logic after input settles.

---

## Section 7 — Web Accessibility (a11y)

**Q22.**
### What is WCAG and what are its four core principles?

WCAG (Web Content Accessibility Guidelines) is the international standard for making web content accessible to people with disabilities. It's published by the W3C and WCAG 2.2 is the current version. Many countries legally require compliance for public-sector websites.

The four principles are often remembered as **POUR**:

- **Perceivable** — users must be able to perceive all information. This means alt text on images, captions on video, sufficient colour contrast (4.5:1 for normal text at AA level).
- **Operable** — all functionality must work via keyboard, not just mouse. No content that flashes more than 3 times per second (seizure risk). Users should have enough time to read content.
- **Understandable** — text should be readable, pages should behave predictably, error messages should explain what went wrong and how to fix it.
- **Robust** — content should work across different browsers, devices, and assistive technologies, now and in the future. This basically means writing valid, semantic HTML.

**Q23.**
### How do ARIA attributes improve accessibility? Give examples.

ARIA (Accessible Rich Internet Applications) attributes tell screen readers things that HTML alone can't communicate — especially for custom interactive widgets. The first rule of ARIA is: *don't use ARIA if a native HTML element can do the job.* A real `<button>` is always better than `<div role="button">` because it's keyboard focusable and operable out of the box.

```
<!-- Dialog / modal: announces role and title to screen reader -->

<div role="dialog" aria-modal="true" aria-labelledby="modal-heading">

<h2 id="modal-heading">Delete account?</h2>

<p>This action is permanent and cannot be undone.</p>

<button autofocus>Cancel</button>

<button>Confirm delete</button>

</div>
```

```
<!-- Accessible icon button (no visible text) -->

<button aria-label="Close menu">

<svg aria-hidden="true" focusable="false">...</svg>

</button>


<!-- Live region: screen reader announces updates without focus moving -->

<div role="status" aria-live="polite" aria-atomic="true">

3 results found

</div>


<!-- Expandable nav item -->

<button aria-expanded="false" aria-controls="submenu-id">Products</button>

<ul id="submenu-id" hidden> ... </ul>
```

**Q24.**

### What is keyboard accessibility and how do you implement it?

- **Use semantic interactive elements.** <button>, <a href>, <input> are focusable and operable by keyboard automatically. A  with an onClick is not.
- **Don't hide the focus ring.** I've seen a lot of codebases with outline: none on everything. That makes the site unusable for keyboard users. Use :focus-visible to show a focus ring only for keyboard navigation without affecting mouse users.
- **Logical tab order.** Tab should move focus in a natural reading order. Avoid tabindex values above 0 — they break the natural flow.
- **Trap focus in modals.** When a modal is open, Tab should cycle through the modal's elements only, not go back to the page behind it.
- **Keyboard shortcuts for custom widgets.** Escape to close dialogs. Arrow keys to navigate menus and list boxes. Enter/Space to activate buttons.

## Section 8 — Git Workflow

**Q25.**

### Describe a feature-branch workflow with PR process.

This is the workflow I follow on my personal projects and the standard I'd expect on a team:

```
# 1. Pull the latest main before starting anything

git checkout main

git pull origin main


# 2. Create a descriptive branch

# I follow: feat/short-description or fix/issue-description

git checkout -b feat/user-login-form


# 3. Work in small, focused commits

# I use Conventional Commits format: type(scope): message

git add src/components/LoginForm.jsx

git commit -m "feat(auth): add login form with email/password validation"


git add src/components/LoginForm.test.jsx

git commit -m "test(auth): add unit tests for login form validation"


# 4. Keep the branch up to date while working

git fetch origin

git rebase origin/main # rebase keeps history linear


# 5. Push and open a PR

git push -u origin feat/user-login-form
```

In the PR I try to include: what this does and why, screenshots for UI changes, any testing steps, and a link to the relevant issue/ticket. On a team I'd expect at least one review approval and passing CI before merging.

**Q26.**

### Explain git rebase vs git merge.

**git merge** creates a merge commit that joins two branches, preserving the full history of when things were branched and merged. The graph looks like a tree with joining lines.

**git rebase** replays your commits on top of the target branch, as if you'd branched off the latest commit. The result is a clean, linear history with no merge commits.

```
# Merge (preserves branch history)

git checkout main

git merge feat/my-feature # creates a merge commit


# Rebase (linear history — looks like the work was done on top of main)

git checkout feat/my-feature

git rebase main # replays my commits on top of latest main

git checkout main

git merge feat/my-feature # now a fast-forward merge, no extra commit
```

I personally prefer rebasing my feature branch before opening a PR — it keeps the git log readable and makes code review easier. But the golden rule is: **never rebase a branch that other people are working on.** Rebasing rewrites commit history, and if a teammate has based their work on your old commits, it creates conflicts that are painful to resolve.

---

## Section 9 — TypeScript

**Q27.**

### What are the main benefits of TypeScript over plain JavaScript?

I was sceptical of TypeScript at first — it felt like extra work. After using it on a bigger project, I won't go back to plain JS for anything serious.

- **Catch bugs before running the code.** TypeScript flags type mismatches, missing properties, and incorrect function arguments at compile time. The number of runtime errors I used to get from typos in property names dropped dramatically.
- **Much better IDE support.** Autocomplete actually knows what properties are available on an object. Renaming a function updates every usage. Go-to-definition works reliably. This saves a lot of time.
- **Code is self-documenting.** A function signature with types tells you exactly what it expects and returns without reading the implementation or looking for comments.

- **Refactoring is safer.** Changing a function signature immediately highlights every broken callsite across the codebase. Without types, you have to run the code and hope your tests catch everything.
- **Gradual adoption.** TypeScript is a superset of JavaScript — you can rename .js files to .ts one at a time and add types incrementally. You don't have to convert everything at once.

**Q28.**

### Explain interfaces vs type aliases, and generics in TypeScript.

Both let you describe the shape of data, but they have different strengths:

```
// Interface — preferred for describing object shapes

// Supports extends and declaration merging

interface User {

id: number;

name: string;

email: string;

role: 'admin' | 'user';

}


interface AdminUser extends User {

permissions: string[];

}


// Type alias — more flexible, can represent anything

type ID = string | number; // union type

type Nullable<T> = T | null; // generic alias

type UserOrAdmin = User | AdminUser; // union of interfaces


// My rule of thumb: interface for objects, type for everything else


// Generics — write code that works for multiple types without losing type safety
```

```
function getFirst<T>(arr: T[]): T | undefined {
return arr[0];
}

getFirst<string>(['a', 'b']); // returns string
getFirst<number>([1, 2, 3]); // returns number

// Generic React component — works with any data type
interface ListProps<T> {
items: T[];
renderItem: (item: T, index: number) => React.ReactNode;
keyExtractor: (item: T) => string;
}

function List<T>({ items, renderItem, keyExtractor }: ListProps<T>) {
return (
<ul>
{items.map((item, i) => (
<li key={keyExtractor(item)}>{renderItem(item, i)}</li>
))}
</ul>
);
}
```

---

## Section 10 — Frontend Architecture

**Q29.**

### What is the difference between CSR, SSR, SSG, and ISR?

These are different strategies for when and where HTML gets generated:

**CSR (Client-Side Rendering)**

The server sends a near-empty HTML shell. JavaScript runs in the browser and builds the UI. This is how traditional React apps (Create React App) work. Fast after initial load, but slow first paint and poor SEO because crawlers see empty HTML.

**SSR (Server-Side Rendering)**

HTML is generated on the server for each request and sent fully formed. Good for SEO and fast initial paint. Works well for pages with user-specific data (dashboards, authenticated content). The downside is server load and TTFB can be slow if data fetching is slow.

**SSG (Static Site Generation)**

HTML is generated once at build time and served as static files from a CDN. Fastest possible delivery — no server work at request time. Perfect for blogs, docs, and marketing pages where content doesn't change per user. Not suitable for real-time or user-specific data.

**ISR (Incremental Static Regeneration) — Next.js specific**

Like SSG, but pages can be regenerated in the background after a set time period, without rebuilding the whole site. Good for e-commerce products or news articles that change occasionally but don't need to be real-time.

```
// Next.js — ISR example

// This page regenerates at most once every 60 seconds

export async function getStaticProps() {

const products = await fetchProducts();

return {

props: { products },

revalidate: 60, // ISR: regenerate after 60 seconds

};

}
```

**Q30.**

### What are micro-frontends and when would you use them?

Micro-frontends apply the microservices idea to the frontend — breaking a large web application into smaller pieces that are independently built, tested, and deployed by separate teams.

The most practical implementation I've learned about is **Module Federation** (Webpack 5), where each micro-frontend exposes components that are loaded at runtime by a shell application without requiring a full rebuild.

**When it makes sense:**

- Large organisations where multiple teams work on the same product and deployments need to be independent
- Teams using different tech stacks that need to coexist in one interface
- Parts of the app that update at very different frequencies (checkout vs help center)

**When to avoid it:**

For small teams or early-stage products, the complexity cost is too high. You need to solve shared state across micro-frontends, shared design systems, routing between them, and consistent CSS — all of which add significant overhead. I would not choose micro-frontends for an intern project or small team.

### Q31.
### What are the SOLID principles and how do they apply to frontend?

These are object-oriented design principles that map surprisingly well to component design:

- **S — Single Responsibility:** Each component should do one thing. A UserCard renders user info. It shouldn't also fetch the user data AND handle routing. Split concerns across components and custom hooks.
- **O — Open/Closed:** Components should be open for extension but closed for modification. Instead of editing a Button component every time you need a variant, accept a variant prop or a render prop to extend it.
- **L — Liskov Substitution:** Implementations should be interchangeable. If you swap one icon library for another, components using it shouldn't break — abstract the specifics behind a consistent interface.
- **I — Interface Segregation:** Don't force a component to accept props it doesn't use. If a component only needs two fields from a large object, pass those two fields, not the whole object.
- **D — Dependency Inversion:** Depend on abstractions, not implementations. Pass a fetchFn prop instead of hard-coding fetch() inside the component — makes testing trivial and lets you swap implementations.

---

*Thank you for the opportunity to apply. I've answered these questions from my own understanding and hands-on experience building projects. Where I've referenced documentation (MDN, React docs, TypeScript handbook), I've tried to synthesise it rather than reproduce it directly. I'm happy to discuss any of these answers in more depth during an interview.*